WEEK 1 · LESSON 6 OF 24 · HANDOUT

Week 1 Project: A Tested Loan-Scoring Service

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=8Zn4hV9xE1I — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCascades 2024: Embracing MLOps — Bridging the Gap Between Machine Learning and DevOps — https://www.youtube.com/watch?v=wRnKcWIZEvk

🎓 **Go deeper:** EuroPython 2020: Best practices for production-ready Docker packaging — https://www.youtube.com/watch?v=H0UPw-ILiz4

Your goal in this lesson: Build a complete, reproducible, tested and packaged prediction service from raw data, and check it against a release checklist.

The brief

The SACCO's IT team asks you for a loan-scoring service they can run on their server. They need: a reproducible model, a validated and versioned API, automated tests, and a Docker package. You will build all of it in this notebook.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

Part 1 — Validate the data

```
LIMITS = {"income_rwf": (10_000, 20_000_000), "loan_rwf": (10_000, 5_000_000), "years_member":
(0, 60),
          "late_payments": (0, 50), "salaried": (0, 1), "momo_tx_month": (0, 1000), "repaid":
(0, 1)}

def validate(df):
    problems = [f"missing column {c}" for c in LIMITS if c not in df.columns]
    for c, (lo, hi) in LIMITS.items():
        if c in df.columns and (df[c].isna().any() or not df[c].between(lo, hi).all()):
            problems.append(f"bad values in {c}")
    return problems

assert validate(applicants) == [], validate(applicants)
print("data OK:", len(applicants), "rows")
```

Part 2 — Reproducible training with a config

```
import hashlib, json
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

CONFIG = {"features": ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"],
          "target": "repaid", "test_size": 0.25, "seed": 0, "C": 1.0, "threshold": 0.5,
"version": "1.0.0"}

def train(config, data):
    tr, te = train_test_split(data, test_size=config["test_size"], random_state=config["seed"],
stratify=data[config["target"]])
    pipe = Pipeline([("scale", StandardScaler()), ("lr", LogisticRegression(C=config["C"],
max_iter=1000))])
    pipe.fit(tr[config["features"]], tr[config["target"]])
    p = pipe.predict_proba(te[config["features"]])[:, 1]
    return pipe, te, {"auc": round(roc_auc_score(te[config["target"]], p), 4),
                      "accuracy": round(accuracy_score(te[config["target"]], p >=
config["threshold"]), 4)}

model, test, metrics = train(CONFIG, applicants)
assert train(CONFIG, applicants)[2] == metrics          # reproducibility check
fingerprint = hashlib.sha256(pd.util.hash_pandas_object(applicants,
index=False).values.tobytes()).hexdigest()[:16]
print(metrics, "| data", fingerprint)
```

Part 3 — Save the artifact with metadata

```
import pathlib, joblib, sklearn
out = pathlib.Path("release"); (out / "model").mkdir(parents=True, exist_ok=True)
joblib.dump(model, out / "model" / "loan_model.joblib")
meta = {**CONFIG, "metrics": metrics, "data_fingerprint": fingerprint, "sklearn_version":
sklearn.__version__}
(out / "model" / "metadata.json").write_text(json.dumps(meta, indent=2))
test.to_csv(out / "model" / "test_data.csv", index=False)
print(sorted(p.name for p in (out / "model").iterdir()))
```

Part 4 — The API

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

class Applicant(BaseModel):
    income_rwf: float = Field(..., ge=10_000, le=20_000_000)
    loan_rwf: float = Field(..., ge=10_000, le=5_000_000)
    years_member: int = Field(..., ge=0, le=60)
    late_payments: int = Field(..., ge=0, le=50)
    salaried: int = Field(..., ge=0, le=1)
    momo_tx_month: int = Field(..., ge=0, le=1000)

app = FastAPI(title="SACCO loan scoring")
scorer = joblib.load(out / "model" / "loan_model.joblib")

@app.get("/health")
def health():
    return {"status": "ok", "model_version": meta["version"]}

@app.post("/predict")
def predict(a: Applicant):
    p = float(scorer.predict_proba(pd.DataFrame([a.model_dump()])[meta["features"]])[0, 1])
    return {"repay_probability": round(p, 3), "approve": p >= meta["threshold"],
"model_version": meta["version"]}

client = TestClient(app)
ok = client.post("/predict", json={"income_rwf": 420_000, "loan_rwf": 300_000, "years_member":
3,
                                   "late_payments": 1, "salaried": 1, "momo_tx_month": 30})
bad = client.post("/predict", json={"income_rwf": 420_000})
print(client.get("/health").json(), ok.status_code, ok.json(), bad.status_code)
```

Part 5 — Automated tests

```
import subprocess, sys
(out / "test_release.py").write_text('''import json, joblib, pandas as pd
from sklearn.metrics import roc_auc_score
meta = json.load(open("model/metadata.json"))
model = joblib.load("model/loan_model.joblib")
data = pd.read_csv("model/test_data.csv")
F = meta["features"]
base = pd.DataFrame([{"income_rwf": 300000, "loan_rwf": 250000, "years_member": 4,
                      "late_payments": 1, "salaried": 1, "momo_tx_month": 35}])
p = lambda df: model.predict_proba(df[F])[:, 1]

def test_quality_gate():
    assert roc_auc_score(data[meta["target"]], p(data)) >= 0.80

def test_late_payments_direction():
    assert p(base.assign(late_payments=4))[0] < p(base)[0]

def test_salary_direction():
    assert p(base.assign(salaried=1))[0] > p(base.assign(salaried=0))[0]

def test_metadata_complete():
    for key in ["version", "features", "metrics", "data_fingerprint", "sklearn_version"]:
        assert key in meta
''')
r = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider",
"test_release.py"],
                   cwd=out, capture_output=True, text=True)
print(r.stdout.strip().splitlines()[-1])
```

Part 6 — Package and check the release

Add `main.py`, `requirements.txt`, `Dockerfile` and `.dockerignore` to `release/` as in Lesson 5. Then run the release checklist:

```
CHECKLIST = {
    "model file": (out / "model" / "loan_model.joblib").exists(),
    "metadata with version and metrics": "metrics" in meta and "version" in meta,
    "quality gate passed": metrics["auc"] >= 0.80,
    "reproducible": train(CONFIG, applicants)[2] == metrics,
    "API validates input": bad.status_code == 422,
    "tests pass": r.returncode == 0,
}
for item, done in CHECKLIST.items():
    print(("[x] " if done else "[ ] ") + item)
print("READY TO SHIP" if all(CHECKLIST.values()) else "NOT READY")
```

Your report (one page)

1. Model: features, metrics, data fingerprint, version.
2. API: endpoints, input limits, example request and answer.
3. Tests: what each test checks and the result.
4. Package: how to build and run it, which settings are environment variables.
5. Risks you already see (we tackle many of them in Weeks 2 and 3).

Marking guide (self-check)

Area	Excellent work
Reproducibility	Config, seed, fingerprint and a passing reproducibility check
API	Validated inputs, versioned answers, health endpoint
Tests	Data, quality and behaviour tests that pass
Packaging	Dockerfile, pinned requirements, no secrets in files
Communication	A clear one-page report

Remember this: A shippable model is validated data, reproducible training, a versioned artifact, a validated API, automated tests and a clean package — checked with a release checklist.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–6 and the report. Then change one thing (a feature, `C`, or the threshold), bump the version to 1.1.0, and run the whole release again. Share your checklist output with your instructor in the lesson Q&A.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What should happen first in the project?
   - A. Write the report
   - B. Validate the data
   - C. Build the Docker image
   - D. Deploy the API
2. How does the project prove training is reproducible?
   - A. By using a GPU
   - B. By guessing
   - C. By deleting the data
   - D. Training twice with the same config gives the same metrics
3. Which file records the version, features and metrics?
   - A. metadata.json
   - B. requirements.txt
   - C. .dockerignore
   - D. Dockerfile
4. What does the API return for a request with only income_rwf?
   - A. 422, because required fields are missing
   - B. A prediction
   - C. 200 with zeros
   - D. 500
5. When is the release 'READY TO SHIP'?
   - A. Always
   - B. When accuracy is 100%
   - C. When the code runs once
   - D. When every checklist item is done
6. What should you change when you retrain with a new setting?
   - A. Nothing
   - B. Only the file name
   - C. The version number, then run the whole release again
   - D. Delete the tests

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.